



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

**UNIVERSITATEA TEHNICĂ DIN
CLUJ-NAPOCA**

Facultatea de Automatică și Calculatoare

**Proiect la disciplina
Limbaje formale si translatoare**

MotoRide CNL Workbench

Student: Traila Sebastian

Grupa: 30235

Specializarea: Calculatoare

An universitar: 2025–2026

Profesor coordonator:

Camelia Mihaela Pinte

Contents

1	Textul Problemei	1
2	Cerintele Aplicatiei	3
2.1	Cerinte functionale	3
3	Complexitatea de Baza	5
3.1	Arhitectura lexer-parser	5
3.2	Intentii si output	5
3.3	Originalitatea proiectului	5
4	Complexitate Avansata Lex/Yacc	7
4.1	Regulile principale ale gramaticii	7
4.2	Tipuri de reguli utilizate	8
5	Adancimea Arborelui de Parsare	9
5.1	Exemplu de arbore de parsare	9
6	Rezolvarea Ambiguitatilor	11
6.1	Normalizarea lexicala multilingva	11
6.2	Normalizarea operatorilor de comparatie	11
6.3	Forme echivalente	11
6.4	Controlled Natural Language	12
7	Avantajele Gramaticii Alese	13
7.1	Gramatica comuna pentru toate limbile	13
8	Scenariul Principal de Test	14
8.1	Test in romana complet - examples/test_ro_plan.txt	14
9	Scenarii Suplimentare de Test	16
9.1	Test in engleza pentru hazard - examples/test_en_hazard.txt	16
9.2	Test multilingv - examples/test_multilingual.txt	16
10	Interfata Web Locala	18
10.1	Arhitectura serverului	18
10.2	Elementele interfetei	18
11	Speech-to-Text	20
11.1	Fluxul de utilizare	20
11.2	Implementare tehnica	20

12 Modulul AI Separat	21
12.1 Pozitionarea modulului AI in arhitectura	21
12.2 Outputul AI Route Advisor	21
12.3 Moduri de functionare	22
13 Structura Proiectului	23
14 Instructiuni de Build si Rulare	24
14.1 Compilare	24
14.2 Rulare din linia de comanda	24
14.3 Rulare interfata web	24
15 Comentarii despre Cod	26
16 Concluzii	27
17 Imbunatatiri Propuse	28

Chapter 1

Textul Problemei

Proiectul **MotoRide CNL Workbench** rezolva problema traducerii automate a comenzilor exprimate in limbaj natural controlat (Controlled Natural Language, CNL) in reprezentari structurate de tip JSON, utilizabile intr-o aplicatie de planificare a traseului moto si de raportare a hazardelor rutiere.

Problema centrala consta in faptul ca utilizatorii doresc sa exprime intentii complexe - alegerea unui traseu, filtrarea dupa preferinte sau raportarea unui pericol rutier - intr-un mod cat mai natural, fie prin tastare, fie prin dictare vocala, fara a fi obligati sa respecte o sintaxa rigida de tip formular.

Totodata, aceste comenzi trebuie sa poata fi exprimate in mai multe limbi (romana, engleza, franceza, spaniola si germana), iar sistemul trebuie sa produca intotdeauna acelasi format de iesire JSON, indiferent de limba utilizata.

Exemple de comenzi suportate

Romana:

Vreau tura relaxata din Brasov pana in Sinaia fara autostrada prefera viraje.

Engleza:

Report gravel on DN1 near Sinaia with high risk.

Germana:

Ich mochte eine entspannte Tour, vermeide Autobahn und ich mochte von Pitesti nach Cluj fahren.

Exemplu de iesire JSON

Pentru comanda de mai sus, aplicatia produce urmatorul JSON:

```
{  
  "intent": "PLAN_RIDE",  
  "start": "Brasov",  
  "destination": "Sinaia",  
  "style": "relaxed",  
  "filters": {  
    "avoidHighways": true,  
    "preferCurves": true  
  }  
}
```

Aceasta reprezentare poate fi integrata direct intr-un sistem de planificare a traseelor sau transmisa catre un API extern de cartografie.

Chapter 2

Cerintele Aplicatiei

2.1 Cerinte functionale

- **Recunoastere multilingva:** aplicatia recunoaste comenzi controlate formulate in romana, engleza, franceza, spaniola si germana.
- **Intentia PLAN_RIDE:** utilizatorul poate specifica un traseu cu punct de plecare, destinatie, stil de condus si filtre optionale.
- **Intentia REPORT_HAZARD:** utilizatorul poate raporta un hazard rutier cu tip, locatie, drum si nivel de risc.
- **Transformare in JSON:** orice comanda valida produce la iesire un obiect JSON structurat.
- **Afisarea arborelui de parsare:** dupa JSON, aplicatia afiseaza un arbore textual al structurii parsate.
- **Filtre pentru PLAN_RIDE:** suport pentru *avoid highways* (evita autostrada) si *prefer curves* (prefera viraje).
- **Conditii meteo:** suport pentru specificarea vizibilitatii, ploii si temperaturii cu operatori de comparatie.
- **Risc:** suport pentru pragul maxim de severitate si activarea unui model ML de evaluare.
- **Interfata web locala:** server Node.js care expune parserul printr-o pagina web accesibila la `http://127.0.0.1:3000`.
- **Speech-to-text:** utilizatorul poate dicta comanda in loc sa o tasteze, browserul (de preferat chrome) preluand vocea.

- **Modul AI separat:** dupa parsarea Lex/Yacc, un modul optional de AI Route Advisor calculeaza scoruri scenice si de risc pe baza JSON-ului produs.

Chapter 3

Complexitatea de Baza

3.1 Arhitectura lexer–parser

Componenta principala a proiectului este perechea **Lex/Flex** (analizor lexical) si **Yacc/Bison** (analizor sintactic). Aceasta arhitectura asigura o separare clara intre recunoasterea cuvintelor individuale si interpretarea structurii comenzii.

Lexerul (fisierul `motoride.1`) citeste textul introdus caracter cu caracter si produce secvente de tokeni. Rolul sau principal in acest proiect este **normalizarea sinonimelor multilingve**: fiecare cuvânt sau expresie din oricare dintre cele cinci limbi suportate este mapat la un token comun, intern. De exemplu, cuvintele *relaxata*, *relaxed*, *detendue*, *relajada* si *entspannt* produc cu totii tokenul `STYLE_RELAXED`.

Parserul (fisierul `motoride.y`) primeste sirul de tokeni si verifica daca acesta respecta una dintre regulile gramaticii definite. Gramatica este scrisa o singura data si este comuna pentru toate limbile, deoarece normalizarea a avut loc deja la nivel de lexer.

3.2 Intentii si output

Aplicatia recunoaste doua intentii principale:

- `PLAN_RIDE` - planificarea unui traseu moto;
- `REPORT_HAZARD` - raportarea unui hazard rutier.

Outputul este intotdeauna JSON valid, urmat de un arbore textual de parsare.

3.3 Originalitatea proiectului

Domeniul ales - planificarea traseului moto - este specific si neconventional pentru un proiect Lex/Yacc. Originalitatea consta in urmatoarele aspecte:

- Comenzile pot fi formulate sau dictate in cinci limbi diferite, iar sistemul le proceseaza fara nicio configurare suplimentara.

- Un singur parser functioneaza pentru toate limbile, deoarece lexerul izoleaza variatia lingvistica.
- Modulul AI este complet separat de parser si nu interfereaza cu procesul de parsare.
- Proiectul include o interfata web si suport pentru speech-to-text, facandu-l usor de demonstrat.

Chapter 4

Complexitate Avansata Lex/Yacc

4.1 Regulile principale ale gramaticii

Gramatica definita in `motoride.y` contine urmatoarele reguli principale:

program Regula de start. Accepta o secventa de una sau mai multe comenzi (statements).

statements Regula recursiva care permite procesarea mai multor comenzi intr-un singur fisier de intrare.

statement O comanda individuala, care poate fi fie o intentie de planificare a unui traseu, fie o raportare de hazard.

plan_marker Marcatorul de inceput al intentiei `PLAN_RIDE`; recunoaste formele echivalente din toate limbile suportate (ex. *vreau tura, I want ride, je veux balade*).

hazard_marker Marcatorul de inceput al intentiei `REPORT_HAZARD`; recunoaste formele de raportare din toate limbile (ex. *raportez, report, signaler, melden*).

plan_parts Lista de parti optionale ale unei comenzi de planificare, procesata recursiv.

plan_part O parte individuala a comenzii de planificare: poate fi o specificatie de ruta, o expresie meteo, o expresie de distanta maxima, o expresie de risc sau o expresie ML.

route_spec Specificatia traseului: punct de plecare (`FROM LOCATION`), destinatie (`TO LOCATION`), stil de condus, si filtrele de evitare a autostrazii sau preferinta pentru viraje.

weather_expr Expresia pentru conditii meteo: vizibilitate, ploi sau temperatura, fiecare cu un operator de comparatie si o valoare numerica.

max_distance_expr Expresia pentru distanta maxima admisa a traseului.

risk_expr Expresia pentru severitatea maxima a riscului acceptat.

ml_expr Expresia care activeaza sau dezactiveaza utilizarea modelului ML de evaluare a riscului.

hazard_part Partile unei comenzi de tip REPORT HAZARD: tipul hazardului, drumul, locatia apropiata si nivelul de severitate.

compare Operator de comparatie normalizat: GT (mai mare decat) sau LT (mai mic decat), utilizat in expresiile meteo.

optional_unit Unitate de masura optionala (km, procente, grade Celsius), care poate lipsi fara a invalida comanda.

4.2 Tipuri de reguli utilizate

- **Reguli recursive** pentru **statements** si **plan_parts**, care permit procesarea mai multor comenzi sau mai multor parti intr-o singura rulare.
- **Reguli optionale** pentru **optional_unit**, care permit comenzi valide indiferent daca utilizatorul specifica sau nu unitatea de masura.
- **Reguli compuse** pentru **route_spec**, care combina mai multe elemente (plecare, destinatie, stil, filtre) intr-o singura constructie.
- **Reguli pentru expresii comparative** pentru **weather_expr**, folosind tokenul normalizat **compare**.
- **Reguli pentru hazard si severitate** in **hazard_part**, care permit specificarea tipului de pericol si a nivelului de risc.
- **Reguli care permit ordine flexibila** in **plan_parts**: utilizatorul poate specifica ruta, vremea, distanta si riscul in orice ordine.

Chapter 5

Adancimea Arborelui de Parsare

Dupa producerea JSON-ului, aplicatia afiseaza in terminal (si in panoul de diagnostice al interfetei web) un **arbore textual de parsare** care reflecta structura completa a comenzii analizate. Arborele permite verificarea vizuala rapida a corectitudinii parsarii.

5.1 Exemplu de arbore de parsare

Pentru comanda:

```
Vreau tura relaxata din Brasov pana in Sinaia fara autostrada prefera viraje
vizibilitate > 5 km ploaie < 30 temperatura intre 10 si 28 C distanta maxim
180 km risc maxim 2 model_ml da.
```

Arborele de parsare afisat este:

```
COMMAND
|-- intent: PLAN_RIDE
|-- route
|   |-- from: Brasov
|   `-- to: Sinaia
|-- preferences
|   |-- style: relaxed
|   |-- avoidHighways: true
|   `-- preferCurves: true
|-- weather
|   |-- visibility: > 5 km
|   |-- rain: < 30 percent
|   `-- temperature: 10..28 C
`-- risk
    |-- useMlModel: true
    `-- maxSeverity: 2
```

Parser output

Copy

```
    "maxSeverity": 2
  }
}

Parse tree:
COMMAND
|-- intent: PLAN_RIDE
|-- route
|   |-- from: Brasov
|   |-- to: Sinaia
|-- preferences
|   |-- style: relaxed
|   |-- avoidHighways: true
|   |-- preferCurves: true
|-- weather
|   |-- visibility: > 5 km
|   |-- rain: < 30 percent
|   |-- temperature: 10..28 C
|-- risk
|   |-- useMLModel: true
|   |-- maxSeverity: 2
```

Figure 5.1: Arbore de parsare afișat în terminal sau în interfața web

Chapter 6

Rezolvarea Ambiguitatilor

6.1 Normalizarea lexicala multilingva

Principalul mecanism de rezolvare a ambiguitatilor este **normalizarea la nivel de lexer**. Cuvintele cu acelasi sens din limbi diferite sunt mapate la acelasi token intern inainte ca parserul sa le proceseze. Astfel, parserul nu vede niciodata cuvinte in limbi specifice, ci doar tokeni abstracti.

Exemple de normalizari:

- *fara, without, sans, sin, ohne* → token WITHOUT
- *din, from, de, desde, von* → token FROM
- *pana la, to, la, hasta, nach* → token TO
- *risc ridicat, high risk, risque eleve, alto riesgo, hohes Risiko* → token HIGH_RISK

6.2 Normalizarea operatorilor de comparatie

Expresiile comparative vorbite sunt de asemenea normalizate:

- *mai mare de, greater than, plus de, mas de, groesser als* → token GT
- *mai mic de, less than, moins de, menos de, kleiner als* → token LT

6.3 Forme echivalente

Anumite constructii sunt acceptate in mai multe ordini sau variante sintactice. De exemplu, *high risk* si *risk high* sunt amandoua forme valide si produc acelasi token. Aceasta flexibilitate este implementata direct in regulile lexerului, nu in gramatica parserului.

6.4 Controlled Natural Language

Este important de precizat ca sistemul nu incearca sa inteleaga limbaj natural liber. El opereaza pe **Controlled Natural Language (CNL)**: utilizatorul trebuie sa foloseasca un vocabular predefinit si o structura relativ fixa. Aceasta alegere reduce drastic ambiguitatea si face sistemul predictibil si testabil, spre deosebire de o abordare bazata exclusiv pe modele de limbaj generative.

Chapter 7

Avantajele Gramaticii Alese

7.1 Gramatica comuna pentru toate limbile

Decizia de a plasa intreaga variatie lingvistica in lexer si de a pastra o gramatica unica in parser aduce mai multe avantaje concrete:

- **Extensibilitate:** adaugarea suportului pentru o noua limba presupune doar extinderea fisierului `motoride.1` cu noi reguli de normalizare. Fisierul `motoride.y` (gramatica) ramane neschimbat.
- **Consistenta outputului:** structura JSON produsa este identica indiferent de limba comenzii, ceea ce simplifica integrarea cu sisteme downstream.
- **Predictibilitate si testabilitate:** gramatica formala garanteaza ca aceeasi comanda produce intotdeauna acelasi rezultat. Testele automate sunt usor de scris si de rulat.
- **Reducerea ambiguitatii:** fata de o traducere libera bazata pe NLP general, CNL limiteaza spatiul de interpretare si elimina ambiguitatea lexicala si sintactica.
- **Transparenta regulilor:** fata de o abordare exclusiv bazata pe AI, Lex/Yacc ofera reguli explicit definite, verificabile si auditabile. Fiecare decizie de parsare poate fi urmarita in gramatica.

Chapter 8

Scenariul Principal de Test

8.1 Test in romana complet - examples/test_ro_plan.txt

Comanda de intrare

Vreau tura relaxata din Brasov pana in Sinaia fara autostrada prefera viraje vizibilitate > 5 km ploaie < 30 temperatura intre 10 si 28 C distanta maxim 180 km risc maxim 2 model_ml da.

JSON produs

```
{
  "intent": "PLAN_RIDE",
  "start": "Brasov",
  "destination": "Sinaia",
  "style": "relaxed",
  "filters": {
    "avoidHighways": true,
    "preferCurves": true
  },
  "weather": {
    "visibility": { "op": ">", "value": 5, "unit": "km" },
    "rain": { "op": "<", "value": 30, "unit": "percent" },
    "temperature": { "min": 10, "max": 28, "unit": "C" }
  },
  "maxDistanceKm": 180,
  "risk": {
    "maxSeverity": 2,
    "useMlModel": true
  }
}
```

}

Explicatia campurilor produse

- **intent:** PLAN_RIDE - intentie de planificare a unui traseu.
- **start:** Brasov - punct de plecare.
- **destination:** Sinaia - destinatie.
- **style:** relaxed - stil de condus relaxat.
- **avoidHighways:** true - evitarea autostrazilor.
- **preferCurves:** true - preferinta pentru drumuri cu viraje.
- **weather:** vizibilitate mai mare de 5 km, ploaie sub 30%, temperatura intre 10 si 28 de grade Celsius.
- **maxDistanceKm:** 180 - distanta maxima admisa.
- **risk:** severitate maxima 2, cu activarea modelului ML.

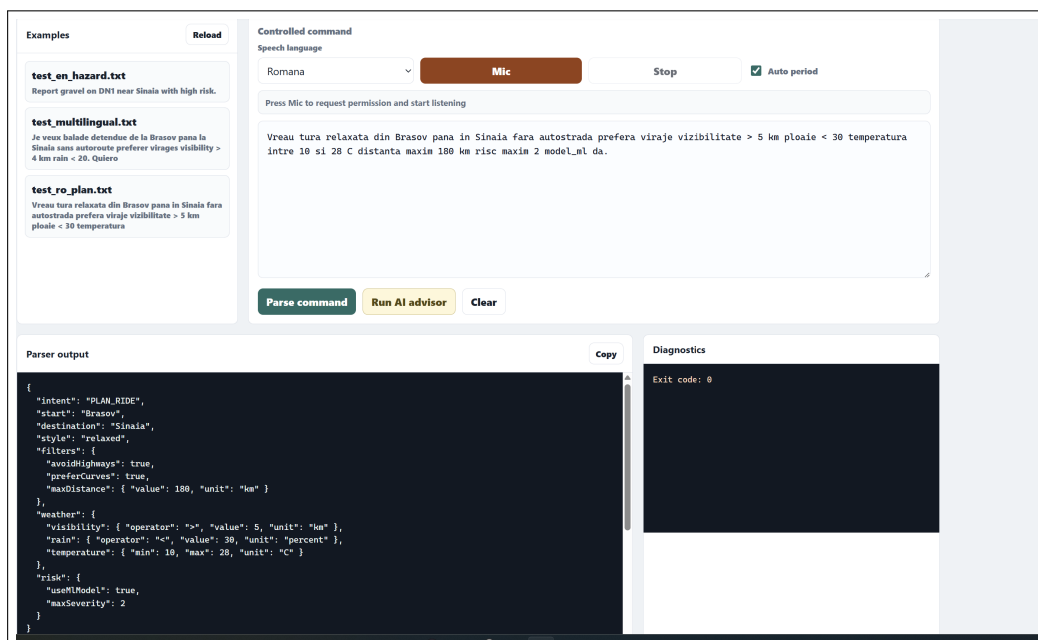


Figure 8.1: JSON pentru testul in romana

Chapter 9

Scenarii Suplimentare de Test

9.1 Test in engleza pentru hazard - examples/test_en_hazard.txt

Comanda de intrare

Report gravel on DN1 near Sinaia with high risk.

JSON produs

```
{
  "intent": "REPORT_HAZARD",
  "hazardType": "gravel",
  "road": "DN1",
  "near": "Sinaia",
  "severity": "high"
}
```

Explicatie

Comanda raporteaza prezenta pietrisului (*gravel*) pe drumul *DN1* in apropierea localitatii Sinaia, cu nivel de risc ridicat. Lexerul recunoaste *report* ca marker de intentie REPORT_HAZARD si *high risk* ca nivel de severitate.

9.2 Test multilingv - examples/test_multilingual.txt

Comenzi de intrare

Franceza:

Je veux balade detendue de la Brasov pana la Sinaia sans autoroute preferer virages visibility > 4 km rain < 20.

Spaniola:

Quiero ruta sport desde Cluj hasta Turda sin autopista preferir curvas
max distancia 70 km.

Franceza (hazard):

Signaler huile sur A3 pres de Turda avec risque eleve.

Germana:

Ich mochte eine entspannte Tour, vermeide Autobahn und ich mochte von
Pitesti nach Cluj fahren.

Observatie

Toate comenzile de mai sus sunt parsate de **aceeasi gramatica**, fara nicio ramura condicionala bazata pe limba. Lexerul normalizeaza *je veux balade*, *quiero ruta* si *ich mochte eine Tour* la acelasi marker de intentie `PLAN_RIDE`, *signaler* la `REPORT_HAZARD`, *sans autoroute* si *sin autopista* si *vermeide Autobahn* la tokenul `WITHOUT HIGHWAY`, si asa mai departe. Din perspectiva parserului, toate aceste comenzi sunt structural identice.

Chapter 10

Interfata Web Locala

Pe langa utilizarea din linia de comanda, proiectul include o interfata web locala care face parserul accesibil printr-un browser obisnuit, fara instalari suplimentare.

10.1 Arhitectura serverului

Interfata este implementata ca un server Node.js minimal (fisierul `ui/server.js`). La primirea unei comenzi din browser, serverul ruleaza executabilul `motoride.exe` cu comanda respectiva si returneaza outputul (JSON si arborele de parsare) catre pagina web.

10.2 Elementele interfetei

Pagina accesibila la adresa `http://127.0.0.1:3000` contine:

- **Textarea pentru comanda** - zona de text in care utilizatorul tasteaza sau dicteaza comanda.
- **Butonul Parse command** - trimite comanda catre server si afiseaza outputul parserului.
- **Butonul Run AI advisor** - ruleaza modulul AI Route Advisor pe baza JSON-ului deja produs.
- **Lista de exemple** - comenzi predefinite selectabile cu un clic, utile pentru demonstratie.
- **Panoul Parser output** - afiseaza JSON-ul produs de Lex/Yacc.
- **Panoul Diagnostics** - afiseaza arborele de parsare textual.
- **Panoul AI Route Advisor** - afiseaza outputul modulului AI (scoruri, recomandari).

LEX/VACC PROJECT
MotoRide CNL Workbench
Parsed successfully

Examples
Reload

test_en_hazard.txt
Report gravel on DN1 near Sinaia with high risk.

test_multilingual.txt
Je veux balade detendue de la Brasov pana la Sinaia sans autoroute preferer virages vizibilitate > 4 km rain < 20. Quiero

test_ro_plan.txt
Vreau tura relaxata din Brasov pana in Sinaia fara autostrada prefera viraje vizibilitate > 5 km ploaie < 30 temperatura

Controlled command
Speech language
Romana
Mic
Stop
Auto period

Press Mic to request permission and start listening

Vreau tura relaxata din Brasov pana in Sinaia fara autostrada prefera viraje vizibilitate > 5 km ploaie < 30 temperatura intre 10 si 28 C distanta maxim 180 km risc maxim 2 model_nl da.

Parse command
Run AI advisor
Clear

Parser output
Copy

```

{
  "intent": "PLAN_RIDE",
  "start": "Brasov",
  "destination": "Sinaia",
  "style": "relaxed",
  "filters": {
    "avoidHighways": true,
    "preferCurves": true,
    "maxDistance": { "value": 180, "unit": "km" }
  },
  "weather": {
    "visibility": { "operator": ">", "value": 5, "unit": "km" },
    "rain": { "operator": "<", "value": 30, "unit": "percent" },
    "temperature": { "min": 10, "max": 28, "unit": "C" }
  },
  "risk": {
    "useModel": true,
    "maxSeverity": 2
  }
}

```

Parse tree:

Diagnostics
Exit code: 0

AI Route Advisor
Not run yet

SCENIC SCORE
-

RISK SCORE
-

RIDE FIT
-

Run the advisor after writing or dictating a valid command.

Figure 10.1: Interfata web completa

Chapter 11

Speech-to-Text

Modulul de speech-to-text permite utilizatorului sa dicteze comanda in loc sa o tasteze, reducand bariera de utilizare pentru demonstratii sau pentru utilizatori mai putin familiarizati cu tastatura (momentan functioneaza numai in chrome/edge).

11.1 Fluxul de utilizare

1. Utilizatorul selecteaza limba dorita din meniu: romana, engleza, franceza, spaniola sau germana.
2. Apasa butonul **Mic** (microfon).
3. Browserul solicita permisiunea de acces la microfon.
4. Utilizatorul vorbeste comanda.
5. Apasa butonul **Stop**.
6. Textul recunoscut apare automat in textarea.
7. Comanda poate fi editata daca este necesar, apoi trimisa la parser.

11.2 Implementare tehnica

Implementarea principala utilizeaza **Web Speech API**, disponibila nativ in Chrome si Edge. Aceasta nu necesita niciun server suplimentar sau API key.

Pentru browserele care nu suporta Web Speech API, exista un fallback care inregistreaza audio si trimite inregistrarea catre **Hugging Face Whisper** pentru transcriere. Aceasta varianta necesita setarea variabilei de mediu `HF_API_TOKEN`.

Este important de subliniat ca **speech-to-text este o extensie de interfata**, nu un inlocuitor al Lex/Yacc. Vocea este convertita in text, iar textul este procesat de parser in mod normal.

Chapter 12

Modulul AI Separat

12.1 Pozitionarea modulului AI in arhitectura

Modulul AI (AI Route Advisor) este complet separat de parser si nu participa la procesul de parsare. Fluxul de date este urmatorul:

Text sau speech → **Lex/Yacc** → JSON → **AI Route Advisor**

AI-ul nu primeste niciodata textul brut al comenzii. El ruleaza exclusiv dupa ce Lex/Yacc a produs un JSON valid si doar pe baza acelui JSON.

12.2 Outputul AI Route Advisor

Pornind de la JSON-ul produs de parser, AI Route Advisor calculeaza si returneaza urmatoarele campuri:

- **scenicScore** - scor de atractivitate scenica a traseului (0–10).
- **riskScore** - scor de risc estimat al traseului (0–10).
- **rideFit** - calificativul general al traseului (de ex. *strong-scenic-fit, moderate-risk*).
- **tags** - etichete descriptive (de ex. *scenic, avoids-highways, curve-friendly, mountain-context*).
- **explanation** - o explicatie textuala a recomandarilor.

Exemplu de output

Pentru traseul Brasov → Sinaia cu filtrele *avoidHighways* si *preferCurves*, AI Route Advisor poate produce:


```

{
  "scenicScore": 8.5,
  "riskScore": 4.2,
  "rideFit": "strong-scenic-fit",
  "tags": ["scenic", "avoids-highways", "curve-friendly", "mountain-context"],
  "explanation": "Ruta Brasov-Sinaia pe drumuri secundare traverseaza zona montana Bucegi, cu potential scenic ridicat si viraje frecvente."
}

```

12.3 Moduri de functionare

AI Route Advisor suporta doua moduri:

- **Local advisor** (implicit, fara API key): bazat pe reguli explicabile definite in `ui/advisor.js`. Nu necesita conexiune la internet.
- **Hugging Face zero-shot classification** (optional): se activeaza daca variabila de mediu `HF_API_TOKEN` este setata. Utilizeaza un model de clasificare de la Hugging Face pentru a evalua traseul.

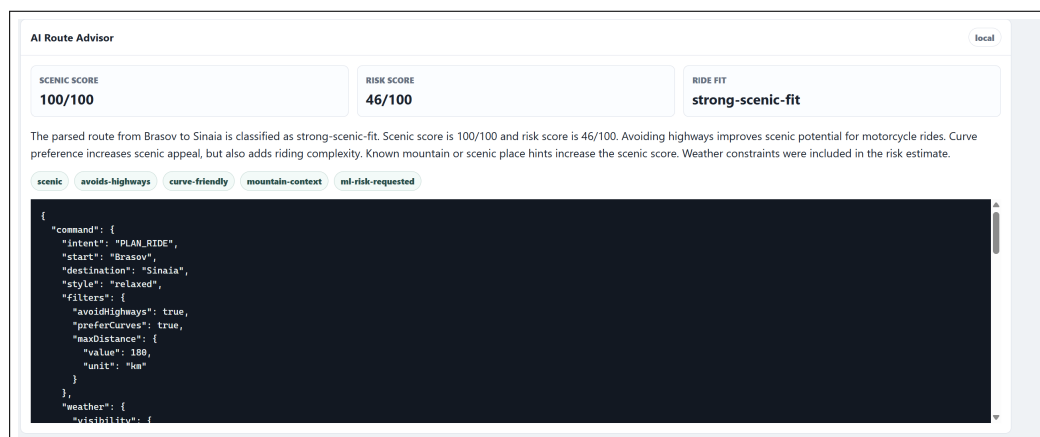


Figure 12.1: Panoul AI Route Advisor cu scoruri si etichete

Chapter 13

Structura Proiectului

```
motoride-cnl-workbench/
|-- motoride.l          # Lexer Flex: normalizare multilingva, tokenizare
|-- motoride.y          # Parser Bison: gramatica CNL, generare JSON si arbore
|-- scripts/
|   `-- build.ps1      # Script PowerShell de compilare (Flex + Bison + GCC)
|-- examples/
|   |-- test_ro_plan.txt    # Test complet in romana
|   |-- test_en_hazard.txt  # Test raportare hazard in engleza
|   `-- test_multilingual.txt # Test cu comenzi in 4 limbi
|-- ui/
|   |-- server.js        # Server Node.js local (ruleaza motoride.exe)
|   |-- app.js           # Logica interfetei web
|   |-- advisor.js       # Modul AI Route Advisor (separat de parser)
|   |-- index.html       # Pagina web principala
|   `-- styles.css       # Stilizarea interfetei
`-- docs/
    `-- autoevaluare.md # Note de autoevaluare si criterii indeplinite
```

Fisierele centrale ale proiectului sunt `motoride.l` si `motoride.y`. Restul fisierelor reprezinta extensii de interfata si infrastructura de build.

Chapter 14

Instructiuni de Build si Rulare

14.1 Compilare

Din directorul radacina al proiectului, se executa scriptul de build PowerShell:

```
cd motoride-cnl-workbench
.\scripts\build.ps1
```

Scriptul ruleaza in ordine: Flex pe `motoride.l`, Bison pe `motoride.y`, apoi GCC pentru a linka fisierele C generate si a produce executabilul `motoride.exe`.

14.2 Rulare din linia de comanda

```
# Test romana complet
Get-Content .\examples\test_ro_plan.txt | .\motoride.exe

# Test raportare hazard engleza
Get-Content .\examples\test_en_hazard.txt | .\motoride.exe

# Test multilingv
Get-Content .\examples\test_multilingual.txt | .\motoride.exe
```

14.3 Rulare interfata web

```
# Pornire server Node.js
node .\ui\server.js

# Deschidere in browser
http://127.0.0.1:3000
```

Serverul porneste automat pe portul 3000. La fiecare cerere de parsare, el ruleaza `motoride.exe` si returneaza outputul catre browser.

Chapter 15

Comentarii despre Cod

Codul sursa al proiectului contine comentarii in engleza, plasate in punctele cheie ale implementarii. Comentariile acopera urmatoarele aspecte:

- **Normalizarea lexicala:** fiecare grup de sinonime din `motoride.l` este insotit de un comentariu care explica ce tokeni produce si din ce limbi provin cuvintele.
- **Regulile principale ale gramaticii:** fiecare regula din `motoride.y` are un comentariu care descrie ce constructie sintactica recunoaste si ce campuri JSON populeaza.
- **Generarea JSON-ului:** actiunile semantice din Bison care construiesc JSON-ul sunt comentate pentru a clarifica maparea intre token si camp JSON.
- **Arborele de parsare:** functia de afisare a arborelui textual este comentata pentru a explica structura recursiva.
- **Separarea AI-ului:** fisierul `ui/advisor.js` contine un comentariu de inceput care explicita ca modulul AI primeste exclusiv JSON de la parser, nu text brut.

Chapter 16

Concluzii

Proiectul **MotoRide CNL Workbench** demonstreaza utilizarea concreta a Lex/Flex si Yacc/Bison pentru implementarea unui translator de Controlled Natural Language intr-un domeniu specific - planificarea traseului moto si raportarea hazardelor rutiere.

Principalele concluzii sunt urmatoarele:

- **Separarea lexer/parser/AI** face aplicatia modulara si extensibila: fiecare componenta are o responsabilitate clara si poate fi modificata independent.
- **Normalizarea multilingva la nivel de lexer** elimina nevoia de a duplica gramatica pentru fiecare limba si reduce semnificativ ambiguitatea.
- **Gramatica formala** ofera predictibilitate si testabilitate ridicate, avantaje pe care o abordare exclusiv bazata pe modele de limbaj generative nu le poate garanta.
- **JSON-ul rezultat** este un format standard, usor de integrat cu aplicatii MotoRide reale, API-uri de cartografie sau alte servicii.
- **Interfata web si speech-to-text** fac proiectul accesibil si usor de demonstrat fara cunostinte tehnice avansate.

Proiectul poate fi extins si imbunatatit in directiile descrise in capitolul urmator.

Chapter 17

Imbunatatiri Propuse

Pe baza experientei acumulate in implementarea proiectului, au fost identificate urmatoarele directii de imbunatatire:

1. **Conectarea la un API real de cartografie** (MotoRide, Mapbox, OpenRouteService): JSON-ul produs de parser ar putea fi trimis direct catre un API de rutare pentru a calcula geometria efectiva a traseului si timpul estimat.
2. **Integrarea unui model ML real de severitate a accidentelor**: in prezent, evaluarea riscului este bazata pe reguli. Un model antrenat pe date reale de accidente rutiere ar creste acuratetea scorului de risc.
3. **Externalizarea sinonimelor intr-un dictionar extern**: sinonimele multilingve din `motoride.1` ar putea fi mutate intr-un fisier de configurare JSON sau YAML, permitand extinderea vocabularului fara recompilare.
4. **Adaugarea de teste automate**: un set de teste de tip input/output expected ar putea fi rulat automat la fiecare build, garantand ca modificarile nu strica parsarea existenta.
5. **Extinderea suportului pentru locatii multi-cuvant**: in prezent, locatiile sunt tokenuri simple. Suportul pentru nume compuse (ex. *Statiunea Busteni*, *Cheile Bicazului*) ar creste acoperirea practica a sistemului.
6. **Folosirea unui model Hugging Face specializat** pentru clasificarea rutelor scenic/risk in locul regulilor explicabile din local advisor, ceea ce ar creste calitatea recomandarilor pentru trasee mai putin cunoscute.